Q1. Which dependencies did you use and why? - Explain why SQLite is necessary to store data locally.

Dependencies used:

* sqflite: This is a plugin that allows Flutter to communicate with the SQLite engine on Android and iOS.

* path: This package provides functions to find and define the correct file paths on the device's storage where the database file will be saved.

Why SQLite is necessary?

* Relational structure: Unlike SharedPreferences (which only stores simple key-value pairs), SQLite is a relational database that allows us to store complex data like "posts" with multiple fields (ID, Title, Content) in an organised table.

* Persistence: It ensures that staff data remains on the device even after the app is closed or the phone is restarted.

* Offline capability: It allows the "offline posts manager" to function entirely without an internet connection, fulfilling the requirement for the media company staff to work anywhere.

Q2. How do you handle database exceptions?

* Database not initialized: We handle this by using try-catch block during the openDatabase call. If the database fails to open (example: due to path), the app catches the exception and can display a user-friendly error message instead of crashing.

* Insert/update/delete errors: We use asynchronous Future methods and await. By wrapping these CRUD operations in try-catch blocks, we can monitor the returned values; for instance if an insert returns 0, we know the operation failed.

* Invalid or corrupted data: We use a Model class (Post) with factory constructor (Post.fromMap). This ensures that any data pulled from the database is validated and correctly typed before it reaches the UI, preventing "null pointer" or "type mismatch" crashes from corrupted rows.

Q3. Explain SQLite in Flutter

* Database vs Table: The database is the entire container (the .db file) that lives on the device's storage. A Table is a specific structure inside that database (like the posts table) that holds the actual rows and columns of data

* CRUD operations: These represent the four essential functions of the app:
  - Create: Adding a new post to the table using db.insert.
  - Read: Retrieving all posts or a specific post details using db.query.

- Update: Modifying an existing post using db.update.

- Delete: Removing a post from the storage using db.delete.

* Asynchronous interaction: Since reading/writing to physical hardware storage is much slower than reading from memory, Flutter interacts with SQLite using async and await. This allows the database to happen in the background so the user interface remains smooth and responsive without freezing.